

Matrix Notation and Dimensions

Deep Learning

Daniel E. Acuna

Associate Professor, University of Colorado Boulder

Contents of This Video

In this video, we will cover:

- Matrix notation for neural network layers
- Understanding weight matrix dimensions
- Batch processing with matrices
- Calculating total parameters in a network
- Why matrix notation enables efficient computation

Single Layer in Matrix Form

For layer l with:

- $n^{(l-1)}$ inputs (neurons in previous layer)
- $n^{(l)}$ outputs (neurons in this layer)

The computation:

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$$

$$\mathbf{a}^{(l)} = \sigma(\mathbf{z}^{(l)})$$

Matrix Dimensions

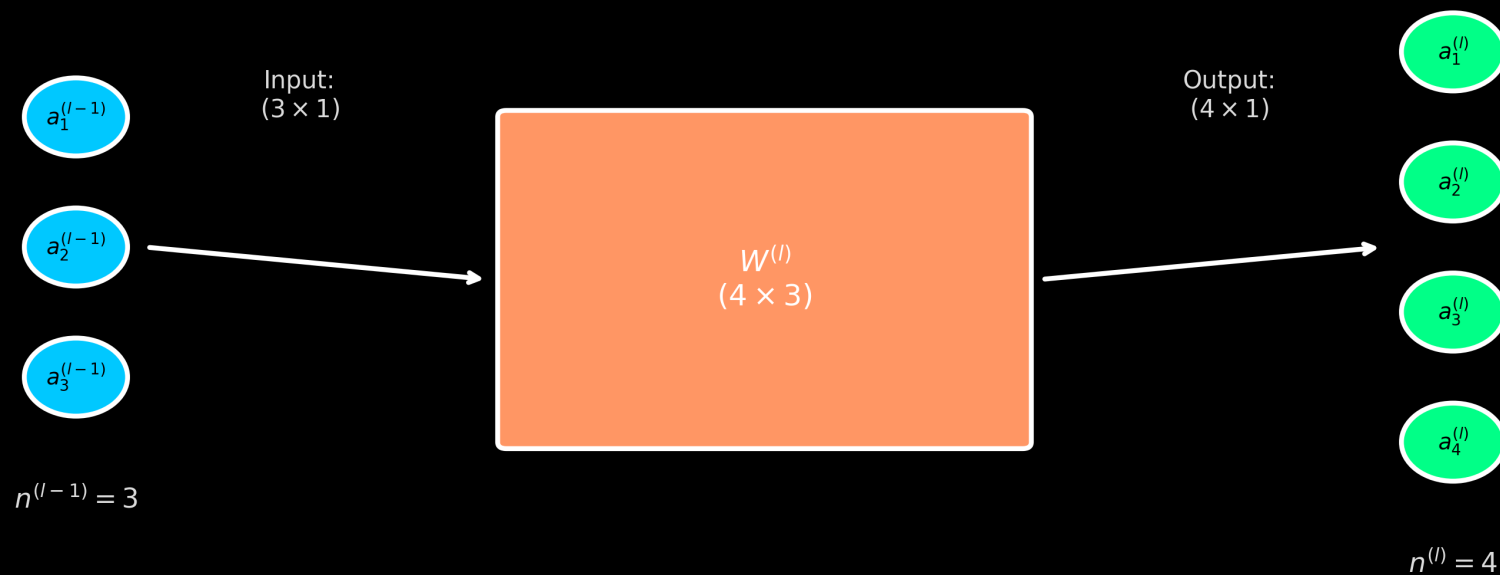
For layer l :

Symbol	Shape	Description
$W^{(l)}$	$(n^{(l)} \times n^{(l-1)})$	Weight matrix
$\mathbf{b}^{(l)}$	$(n^{(l)} \times 1)$	Bias vector
$\mathbf{a}^{(l-1)}$	$(n^{(l-1)} \times 1)$	Input activations
$\mathbf{a}^{(l)}$	$(n^{(l)} \times 1)$	Output activations

Key rule: Weight matrices are (outputs $\times$ inputs)

Dimension Rule: Outputs × Inputs

Weight Matrix Dimensions: (outputs × inputs) = (4×3)



Matrix Multiplication Check

$$\underbrace{W^{(l)}}_{(n^{(l)} \times n^{(l-1)})} \times \underbrace{a^{(l-1)}}_{(n^{(l-1)} \times 1)} + \underbrace{b^{(l)}}_{(n^{(l)} \times 1)} = \underbrace{z^{(l)}}_{(n^{(l)} \times 1)}$$

Example with numbers:

$$(4 \times 3) \times (3 \times 1) + (4 \times 1) = (4 \times 1) \checkmark$$

The inner dimensions must match!

$$(4 \times \boxed{3}) \times (\boxed{3} \times 1) = (4 \times 1)$$

Batch Processing

Single sample: $\mathbf{a}^{(l-1)}$ is $(n^{(l-1)} \times 1)$

Batch of B samples: $A^{(l-1)}$ is $(n^{(l-1)} \times B)$

$$\mathbf{Z}^{(l)} = \mathbf{W}^{(l)} A^{(l-1)} + \mathbf{b}^{(l)}$$

Dimensions:

$$(n^{(l)} \times n^{(l-1)}) \times (n^{(l-1)} \times B) = (n^{(l)} \times B)$$

Key benefit: All samples processed in parallel!

Concrete Example: MNIST Network

Architecture: $784 \rightarrow 128 \rightarrow 10$

- Input: 28×28 pixel images = 784 features
- Hidden layer: 128 neurons with ReLU
- Output layer: 10 neurons (digits 0-9) with softmax

Weight dimensions:

- $W^{(1)}: (128 \times 784)$
- $\mathbf{b}^{(1)}: (128 \times 1)$
- $W^{(2)}: (10 \times 128)$
- $\mathbf{b}^{(2)}: (10 \times 1)$

Parameter Count

Total parameters = sum of all weights and biases

Layer 1:

- Weights: $128 \times 784 = 100,352$
- Biases: 128

Layer 2:

- Weights: $10 \times 128 = 1,280$
- Biases: 10

Total: $100,352 + 128 + 1,280 + 10 = \boxed{101,770}$ **parameters**

General Parameter Formula

For a network with layers of sizes:

$$n^{(0)} \rightarrow n^{(1)} \rightarrow n^{(2)} \rightarrow \dots \rightarrow n^{(L)}$$

Parameters in layer l :

$$\text{Weights: } n^{(l)} \times n^{(l-1)}$$

$$\text{Biases: } n^{(l)}$$

$$\text{Total for layer } l: n^{(l)} \times (n^{(l-1)} + 1)$$

Total parameters:

$$\sum_{l=1}^L n^{(l)} \times (n^{(l-1)} + 1)$$

Why Matrix Notation Matters

Conceptual clarity:

- Compact representation
- Easy to trace data flow
- Clear dimension requirements

Computational efficiency:

- Enables batch processing
- GPU-optimized operations
- Parallelization across samples

All deep learning is built on matrix operations at its core

What We've Covered

In this video, we've learned:

- Weight matrices have shape (outputs $\times$ inputs)
- For layer l : $W^{(l)}$ is $(n^{(l)} \times n^{(l-1)})$
- Batch processing: $(n^{(l)} \times n^{(l-1)}) \times (n^{(l-1)} \times B) = (n^{(l)} \times B)$
- Parameter count: $\sum n^{(l)} \times (n^{(l-1)} + 1)$
- Matrix operations enable efficient GPU computation